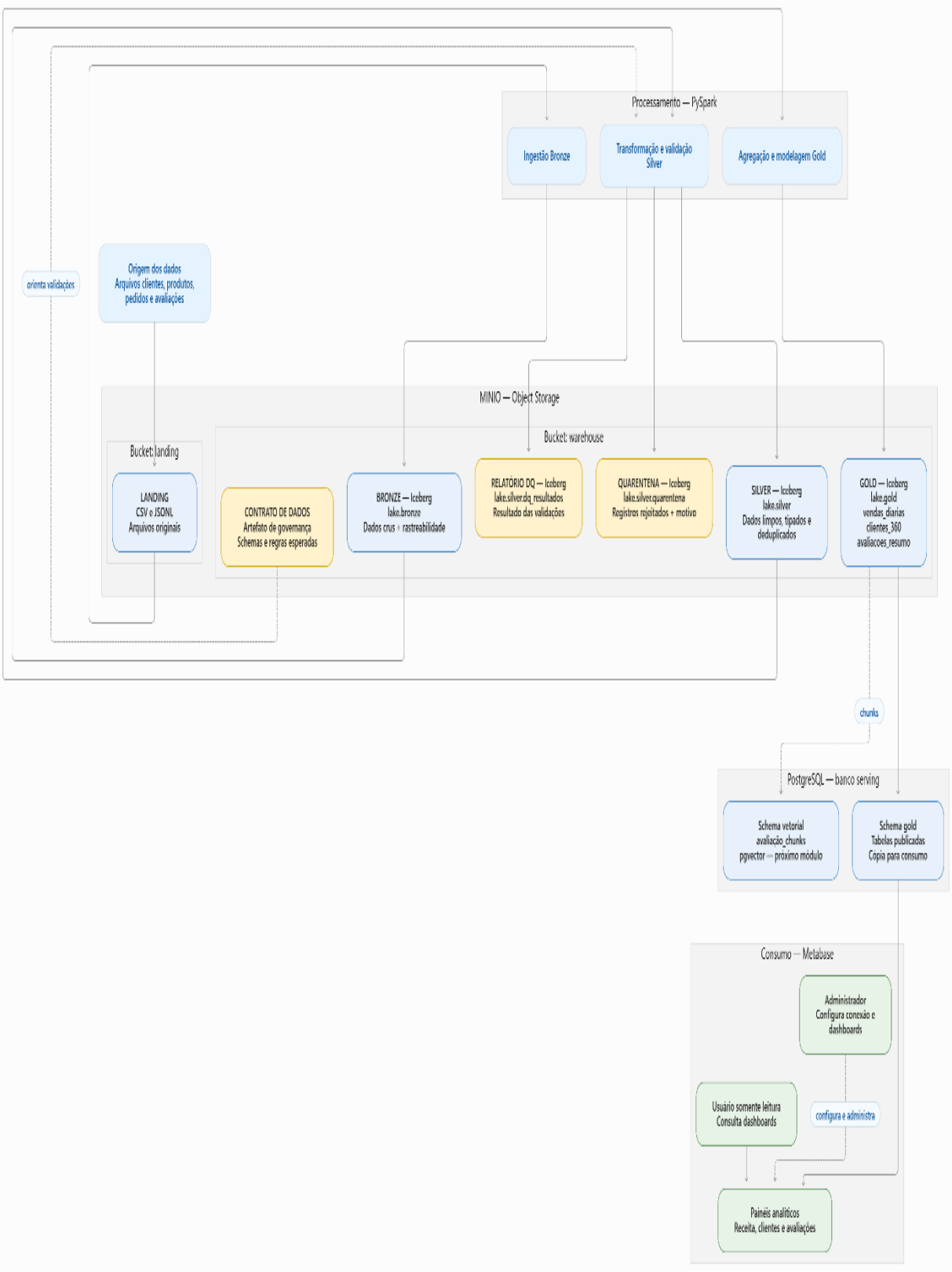
Arquitetura



2. Explicação de cada etapa

Etapa	O que acontece	Entrada → Saída
Landing	Recebe os arquivos originais, sem transformação.	CSV/JSONL → arquivos no bucket landing.
Bronze	O PySpark lê os arquivos e registra os dados no Iceberg, acrescentando informações de rastreabilidade.	CSV/JSONL → tabelas Iceberg.
Silver	Aplica limpeza, conversão de tipos, padronização, deduplicação e regras de qualidade.	Bronze → tabelas Silver tratadas.
Quarentena	Armazena os registros que violam regras, identificando o motivo da rejeição.	Registros inválidos → tabela lake.silver.quarentena.
DQ	Registra os resultados das verificações de qualidade, indicando OK, AVISO ou FALHA.	Tabelas Silver → lake.silver.dq_resultados.
Gold	Constrói tabelas analíticas e agregações para os diferentes consumidores.	Silver → tabelas Gold em Iceberg.
PostgreSQL	Recebe as tabelas Gold para disponibilização.	Iceberg → tabelas no schema gold.
Metabase	Consulta o PostgreSQL para construir gráficos e dashboards.	PostgreSQL → informações visuais.

3. Organização dos buckets e formatos

O MinIO possui dois buckets principais:

Bucket: landing

Arquivos recebidos originalmente

- mercadonordeste/lote1/clientes.csv
- mercadonordeste/lote1/produtos.csv
- mercadonordeste/lote1/pedidos.jsonl
- mercadonordeste/lote1/avaliacoes.csv

Bucket: warehouse

Armazenamento das tabelas Iceberg

- lakehouse/bronze/
- lakehouse/silver/
- lakehouse/gold/

Cada tabela possui seus arquivos de dados e metadados Iceberg.

Camada	Formato
Landing	CSV e JSONL
Bronze	Iceberg (dados em Parquet)
Silver	Iceberg (dados em Parquet)
Gold	Iceberg (dados em Parquet)
PostgreSQL	Tabelas relacionais no schema gold

4. Quarentena, DQ e contrato de dados

Esses três elementos possuem funções diferentes:

- Quarentena: recebe os registros que não passaram pelas regras de transformação. Exemplo: pedido cujo cliente não existe.
- DQ (Data Quality): armazena os resultados das verificações de qualidade na tabela lake.silver.dq_resultados.
- Contrato de dados: define a estrutura esperada dos dados, como campos obrigatórios, tipos, formatos e regras de negócio.

Uma observação importante: o notebook implementa a quarentena e o relatório DQ. O contrato de dados não aparece implementado explicitamente nos três notebooks. No diagrama, ele está representado como um artefato de governança que deveria orientar as validações.

5. Quem escreve e quem somente lê?

É importante separar as permissões de processamento das permissões de consumo.

Componente / usuário	Permissão	Responsabilidade
PySpark / administração	Escrita e leitura	Ingestão, transformação e gravação das camadas.

MinIO / Iceberg	Conforme credenciais do serviço	Armazenamento dos arquivos e tabelas.
Publicador PostgreSQL	Escrita	Atualiza as tabelas Gold no PostgreSQL.
Administrador Metabase	Administração	Configura conexão, perguntas e dashboards.
Usuário somente leitura	Apenas leitura	Consulta dashboards e indicadores.

Atenção: o notebook demonstra a criação do administrador do Metabase e a conexão com PostgreSQL, mas não implementa explicitamente um usuário de somente leitura. Essa permissão deverá ser configurada na implantação.

O ideal é que o usuário do Metabase tenha apenas permissão SELECT nas tabelas do schema gold, sem acesso de escrita ou alteração dos dados.

6. Respostas: o que entra, o que sai e qual a regra principal?

O que entra?

Quatro conjuntos de arquivos do Mercado Nordeste: clientes, produtos, pedidos e avaliações, nos formatos CSV e JSONL. Eles chegam à Landing e são ingeridos na Bronze preservando os dados originais e adicionando informações de rastreabilidade.

O que sai?

Três tabelas analíticas principais são produzidas na Gold:

- vendas_diarias: informações de vendas e receitas por período.
- clientes_360: visão consolidada dos clientes.
- avaliacoes_resumo: resumo das avaliações.

Essas tabelas são armazenadas no Iceberg e publicadas no PostgreSQL, de onde o Metabase realiza as consultas para construir os dashboards.

Qual é a regra principal da arquitetura?

A regra central é garantir que nenhum dado seja perdido silenciosamente durante o processamento.

Na Silver, isso é representado pela reconciliação:

\[\text{Entrada} = \text{Saída válida} + \text{Quarentena} \]

Além disso:

- A Bronze preserva os dados originais.

- A Silver garante a qualidade e a integridade.
- A Gold disponibiliza os dados preparados para consumo.
- O Iceberg permanece como fonte oficial da verdade.
- O PostgreSQL funciona como camada de disponibilização para o Metabase.

Em resumo: a arquitetura transforma arquivos brutos em informações confiáveis para análise, mantendo rastreabilidade, controle de qualidade e separação entre quem administra os dados e quem apenas consome os indicadores.

Respostas

Qual regra da Silver mandou mais registros para a quarentena e por quê?

A regra da Silver que enviou mais registros para a quarentena foi pedidos_cliente_orfao, com 40 registros. Isso aconteceu porque esses pedidos possuíam um cliente_id que não existia na tabela de clientes da Bronze. A validação de integridade referencial entre pedidos e clientes foi a que mais rejeitou registros.

Se a reconciliação não fechasse, o que você investigaria primeiro?

A diferença entre a quantidade de registros de entrada (Bronze) e a quantidade de registros tratados na Silver e enviados para a quarentena.

Na seguinte ordem:

1. Duplicidades: verificaria se a deduplicação por pedido_id e avaliacao_id descartou a quantidade esperada de registros.

2. Quarentena: conferiria se todos os registros rejeitados pelas regras de qualidade foram contabilizados corretamente e se não houve perda de registros durante o processo.

3. Integridade referencial: verificaria se os pedidos com cliente_id inexistente foram corretamente enviados para a quarentena.

4. Explosão dos itens: verificaria a transformação da coluna itens em itens_pedido, pois um pedido pode gerar várias linhas. Essa operação altera a granularidade e não pode ser comparada diretamente com a quantidade de pedidos.

5. Gravação dos dados: verificaria se todas as tabelas da Silver e da quarentena foram gravadas corretamente, sem falhas ou duplicações.

Que dado pessoal existe na sua arquitetura e como ele foi tratado (mascarado, removido, restrito)?

Na arquitetura Medalhão, o principal dado pessoal identificado está na tabela clientes, especialmente o endereço de e-mail. Não foi feito nenhum tratamento.

O que você mudaria nesta arquitetura se o volume de dados fosse 1.000 vezes maior?

Se o volume de dados fosse 1.000 vezes maior, eu manteria a arquitetura Medalhão (Bronze, Silver e Gold), mas faria mudanças principalmente na infraestrutura de processamento, armazenamento e escalabilidade.

Considerando a arquitetura atual, que utiliza PySpark em modo local, MinIO e Apache Iceberg, faria as seguintes mudanças:

1. Processamento distribuído

- Substituiria o Spark em modo local por um cluster distribuído.
- Utilizaria vários nós de processamento para dividir as tarefas.
- Implementaria processamento paralelo e particionamento dos dados.
- Avaliaria o uso de Kubernetes para gerenciamento dos recursos.

2. Armazenamento

- Manteria o conceito de Object Storage, mas utilizaria uma infraestrutura escalável, como Amazon S3, OCI Object Storage ou MinIO distribuído.
- Manteria o Apache Iceberg para gerenciamento das tabelas, aproveitando recursos como evolução de schema, particionamento e controle de versões.
- Utilizaria formatos colunares, como Parquet, para otimizar consultas e reduzir custos de armazenamento.

3. Otimização das camadas Medalhão

- Bronze: ingestão incremental, evitando reprocessar todos os dados a cada execução.
- Silver: processamento distribuído, deduplicação otimizada e execução das regras de qualidade em paralelo.
- Gold: agregações pré-calculadas e tabelas otimizadas para atender às consultas analíticas.

4. Orquestração

Implementaria uma ferramenta como Apache Airflow para coordenar os pipelines, controlar dependências, monitorar falhas e permitir novas tentativas de execução.

5. Monitoramento e qualidade

Criaria mecanismos de monitoramento para acompanhar:

- Tempo de processamento.
- Volume de dados ingeridos.
- Registros rejeitados na quarentena.
- Consumo de memória e CPU.
- Falhas e atrasos nos pipelines.

Eu não mudaria o conceito da arquitetura Medalhão, pois ele continua adequado para grandes volumes de dados. Mudaria a infraestrutura para permitir processamento distribuído, ingestão incremental, armazenamento escalável e monitoramento automatizado.

O principal objetivo seria garantir que o crescimento do volume de dados não provocasse aumento proporcional no tempo de processamento e nos custos da infraestrutura.